NO CAP

Backend Engineering Blueprint

Design it. Break it. Scale it. — for one learner, at zero cost.

DOCUMENT	**Backend Engineering Blueprint**
VERSION	**v2.1 (personal app, zero-cost)**
DATE	**16 August 2026**
RUNTIME	**Cloudflare Worker (Python/FastAPI on Pyodide)**
DATABASE	**Cloudflare D1 (SQLite), R2, KV, Durable Objects**
STATUS	**Working spec**

PRODUCT THESIS

A personal system-design workbench that turns passive reading into a repeatable loop of learning, visualization, reasoning, simulation, architecture practice, and interview readiness — deployable for zero cost, forever, on Cloudflare's genuine free tier, with hard quota guards instead of accidental billing.

What Changed: v2.0 to v2.1

v2.0 proposed a FastAPI server on a managed VM, PostgreSQL, Redis, Celery workers, paid TTS/STT adapters, Firecracker code execution, and live cloud pricing APIs. v2.1 rewrites the entire backend for Cloudflare's free tier: FastAPI on Python Workers (Pyodide), D1 (SQLite), R2, KV, Durable Objects, Queues, and Workers AI. Every adapter has a deterministic fallback. Every optional service has a quota guard.

Backend Pivots

- FastAPI on managed VM → FastAPI on Python Workers (Pyodide).
- PostgreSQL → D1 (SQLite). Schema simplified; user_id omitted in MVP.
- Redis → removed. Caching via browser/edge/D1; KV is OPTIONAL enrichment only (not core).
- Celery/RQ → Cloudflare Queues + Cron Triggers.
- S3 → R2 (free egress).
- Yjs WebSocket server → Durable Objects + Yjs CRDT.
- TTS adapter (multi-provider) → browser Web Speech API. Server stores metadata only.
- STT adapter (multi-provider) → browser Web Speech API. Server stores transcript only.
- Code executor (Firecracker/e2b) → DEFERRED. Code Walkthrough Mode replaces it.
- Cloud pricing service (live APIs) → bundled pricing JSON in git.
- Calendar adapter (Google/MS) → DEFERRED to Tier C.

Schema & API Changes

- user_id omitted from MVP tables (single user). Added back in v2.0 if multi-user.
- New tables: quota_usage (Tier A), notes/annotations/highlights/collections/collection_items/cost_estimates/career_paths (Tier B), voice_sessions/voice_interview_attempts/collab_sessions/collab_participants (Tier C/B v2.0).
- Content tables REMOVED from D1. Content lives as versioned JSON in git, served as static bundles.
- Mastery dimensions reduced from 7 to 5 in MVP. Cost + Code dimensions added in v1.0.
- Quota guard middleware wraps every optional service call. Raw quota data under Settings → System Health (not prominent in main UI).
- Worker constraint: thin Python only (no NumPy/Pandas/SciPy/native deps). Heavy compute runs client-side.
- AI model registry selects free-compatible models (some now require Workers Paid).
- AuthProvider abstraction (GitHub now, Google future).
- /quota/today endpoint added (Tier A).
- Auth simplified: GitHub OAuth, single user. No roles system.
- Code execution endpoints REMOVED. /patterns/[slug] returns static JSON only.
- /cost/estimate uses bundled pricing JSON, no external API call.
- /voice/sessions stores metadata only; audio handled client-side.

IMPLEMENTATION STANCE

The backend is a single Cloudflare Worker with clear module boundaries. No microservices, no Kubernetes, no event streaming. The static-first content architecture means most requests never hit the Worker at all — only mutations and user-specific reads do. This keeps the 100k Worker requests/day free tier effectively unlimited for a single user.

1. Backend Goals

· Serve one coherent domain model to the web/PWA client. (Single user; no multi-tenant complexity in MVP.)

· Keep user progress durable and auditable. Notes, annotations, and personal architectures are never silently lost.

· Content is static (versioned JSON in git). D1 stores user state only. Zero DB queries for content reads.

· Power deterministic simulations and architecture evaluation entirely client-side. Zero Worker requests per simulation/validation.

· Support optional AI, voice, and link-preview integrations without making them critical path. Each has a deterministic fallback.

· Remain a single Cloudflare Worker with clear module boundaries. No premature extraction.

· Provide a complete quota-guard layer so failure of any external service is visible, logged, and recoverable — never billed.

· Cost: Rs 0/month forever, no trial credits, no credit card, no surprise bills.

2. Recommended Repository Layout

```
nocap/
  apps/
    web/                        # Next.js (static export) -> Cloudflare Pages
      app/
      components/
      public/
      service-worker.js         # PWA
      next.config.js            # output: 'export'
    worker/                     # FastAPI on Python Workers -> Cloudflare Worker
      src/
        main.py                 # FastAPI app entry
        api/
          auth.py               # GitHub OAuth
          progress.py           # /progress, /events/batch
          attempts.py           # quiz/sim/architecture attempts
          interviews.py         # interview sessions
          notes.py              # Tier B
          collections.py        # Tier B
          cost.py               # Tier B (uses bundled pricing)
          career.py             # Tier B
          voice.py              # Tier C (metadata only)
          collab.py             # Tier B (v2.0) - Durable Object client
          mentor.py             # Tier C - Workers AI + quota guard
          quota.py              # Tier A - /quota/today
        core/
          config.py
          auth.py
```

```
        logging.py
        dependencies.py
        quota_guard.py          # NEW - wraps every optional service
      domain/
        users/
        learning/
        assessment/
        architecture/
        notes/                  # Tier B
        collections/            # Tier B
        cost/                   # Tier B
        career/                 # Tier B
        voice/                  # Tier C
        collab/                 # Tier B (v2.0)
      db/
        schema.sql              # D1 schema
        migrations/
        repositories/
      adapters/
        workers_ai.py           # Cloudflare Workers AI (Tier C)
        link_preview.py         # Worker fetch + KV cache (Tier C)
      do/                       # Durable Objects (Tier B v2.0)
        collab_session.py
    wrangler.toml               # Worker config + D1/R2/KV/DO bindings
    migrations/                 # D1 SQL migrations
  content/                      # Static content (versioned JSON in git)
    concepts/
    simulations/
    case-studies/
    pricing/                    # Bundled cloud pricing dataset
    code-patterns/
  packages/                     # Shared packages (monorepo)
    types/                      # TypeScript types shared web <-> worker
    content-schema/             # Zod schemas for content JSON
    design-tokens/              # Tailwind config + design tokens
  .github/
    workflows/
      deploy.yml                # GitHub Actions -> wrangler deploy
```

v2.1 monorepo layout. apps/web on Pages, apps/worker on Workers.

3. Module Responsibilities

Module	Responsibilities	Tier
content (static)	Concepts, lessons, quizzes, simulations, case studies, code patterns, pricing data. Served from Pages, not Worker.	A
learning	Progress, mastery, recommendations, review queue, streaks. D1-backed.	A
assessment	Attempts, scoring. D1-backed. Scoring runs on Worker (small, fast).	A
architecture	Saves/evaluates architecture attempts. Validation runs client-side; Worker stores results.	A
simulation	Simulation definitions are static. Attempts stored in D1. Engine runs client-side.	A
interviews	Session state, prompts, timing, rubric scoring. D1-backed.	A (text) / C (voice)
integrations	Workers AI (quota-guarded), link previews (KV-cached).	C
analytics	Event ingestion and aggregation. D1, partitioned by month.	A
quota	Tracks daily Worker/D1/R2/KV/AI usage. Reset by Cron. Exposed via /quota/today.	A
admin/content	NOT a runtime module. Content edits are PRs to the content/ directory in git.	-
notes	Note CRUD, annotation CRUD, highlight CRUD, knowledge graph generation, flashcard conversion.	B
collections	Collection CRUD, item ordering, system starter packs (static).	B
cost	Cost estimate calculation using bundled pricing JSON. Comparison across clouds. Estimate revalidation on pricing dataset change.	B
career	Role profiles (static), gap analysis, 4-week plan generation.	B
voice	TTS session metadata (audio is client-side via Web Speech API). Voice interview transcripts (STT is client-side).	C
collab	Durable Object for CRDT sync + presence. R2 snapshots on save.	B (v2.0)

4. Core Database Schema (D1 / SQLite)

v2.1 schema is dramatically simpler than v2.0. **Content tables are removed** (content is static JSON in git). **user_id is omitted from MVP tables** (single user). New tables added incrementally per tier. SQLite syntax (D1 is SQLite, not PostgreSQL).

Table	Key fields	Notes
users	id, github_id, email, timezone, preferences_json, target_role, created_at	Single row in MVP (id=1). github_id is the stable identity key.
mastery	concept_slug, learn_score, recall_score, apply_score, explain_score, interview_score, updated_at	No user_id in MVP. v1.0 adds cost_score, code_score. Current state only.
review_items	concept_slug, due_at, interval_days, ease, repetitions, source_type, source_id	source_type: concept | flashcard (converted from note). No user_id in MVP.
learning_events	id, type, concept_slug, payload_json, created_at	Append-only. Partitioned by month via D1 view. 12-month retention.
attempts	id, type, ref_id, response_json, score, concept_version, created_at	type: quiz | simulation | architecture. ref_id points to the static content. Immutable.
interview_sessions	id, mode, prompt_slug, state_json, score_json, created_at	mode: text | voice. State is compact JSON.
achievements	slug, title, description, category, tier	Static definition (in git). D1 stores only user_achievements.
user_achievements	achievement_slug, earned_at, payload_json	Earned achievement record. No user_id in MVP.
quota_usage	date, service, count, last_updated	Composite PK (date, service). Reset by Cron at midnight UTC. Services: worker, d1_read, d1_write, r2_class_a, r2_class_b, kv_read, kv_write, do_request, ai_neurons.
notes [B]	id, concept_slug, body_md, created_at, updated_at	Personal concept-level notes. Markdown.
annotations [B]	id, concept_slug, concept_version, block_id, anchor_json, body_md, created_at	Margin annotations anchored to a specific paragraph or diagram element.
highlights [B]	id, concept_slug, concept_version, block_id, selection_json, category, created_at	Text selections. category: 0-3 (learner-defined).

Table	Key fields	Notes
collections [B]	id, name, goal, slug, created_at	User-defined study lists.
collection_items [B]	collection_id, concept_slug, order_index, added_at	Concept-in-collection with ordering.
cost_estimates [B]	id, architecture_hash, cloud, region, breakdown_json, total_monthly_usd, pricing_dataset_version, as_of_date, created_at	Saved cost calculations. architecture_hash enables dedup.
career_paths [B]	id, target_role, gap_analysis_json, plan_json, generated_at	User career plans. Regenerable on demand.
voice_sessions [C]	id, concept_slug, voice_choice, playback_position_sec, completed_chapters_json, created_at	TTS session metadata. Audio NOT stored (browser TTS).
voice_interview_attempts [C]	id, interview_session_id, transcript_text, clarity_score, created_at	Voice interview transcripts. Audio NOT stored (browser STT).
collab_sessions [B v2.0]	id, owner_user_id, crdt_r2_key, started_at, ended_at, invite_url_slug	Collab session metadata. invite_url_slug is 32-byte random.
collab_participants [B v2.0]	session_id, user_id, joined_at, left_at, cursor_color	Per-user participation log. cursor_color assigned on join.

5. Important Constraints and Indexes

- Unique users.github_id.
- Unique mastery.concept_slug (in MVP; composite with user_id in v2.0).
- Index review_items(due_at) WHERE due_at IS NOT NULL - fast "what's due" query.
- Index learning_events(created_at) - partitioned by month.
- Index attempts(type, ref_id, created_at) - fast "my attempts for this concept" query.
- Index attempts(concept_version) - fast "attempts against this version" query.
- Index quota_usage(date, service) - fast "today's usage" query.
- Unique notes(concept_slug) - one note per concept per user (in MVP).
- Index annotations(concept_slug, block_id) - fast lookup within a lesson.
- Index highlights(concept_slug, block_id).
- Unique collections(slug) - per-user collection slugs (in MVP).
- Index collection_items(collection_id, order_index).
- Index cost_estimates(architecture_hash, cloud, region) - dedup lookup.
- Index career_paths(target_role).
- Index voice_sessions(concept_slug) - "have I listened to this before?".
- Index voice_interview_attempts(interview_session_id).
- Unique collab_sessions(invite_url_slug) - unguessable invite URLs.
- Index collab_participants(session_id, user_id).
- **Note:** D1 uses SQLite, which supports CREATE INDEX but not partial indexes directly. Use a computed column or view for "due_at IS NOT NULL" optimization if needed.

6. REST API Contract

v2.1 splits the API: **content endpoints are static** (Next.js static props, served from Pages, zero Worker cost) and **user state endpoints are dynamic** (Worker + D1). The contract is intentionally small — only mutations and user-specific reads hit the Worker.

Method	Endpoint	Purpose	Tier
GET	/auth/login	Initiate GitHub OAuth flow	A
GET	/auth/callback	OAuth callback	A
GET	/auth/me	Current user info	A
POST	/auth/logout	Clear session	A

Method	Endpoint	Purpose	Tier
GET	/concepts/[slug]	Concept content + metadata (Next.js static props from JSON)	A
GET	/tracks	Track list (static)	A
GET	/concepts/[slug]/related	Prereqs, related, contrasts (static)	A
GET	/case-studies/[slug]	Case study (static)	A
GET	/patterns/[slug]	Code pattern (static, no execution)	B
GET	/v1/home	Daily dose, review queue, recommended next actions	A
GET	/v1/concepts/[slug]/mastery	User's mastery for this concept	A
POST	/v1/learning/events	Batch learning events	A
GET	/v1/reviews	Due review items	A
POST	/v1/reviews/[id]/answer	Record recall result	A
POST	/v1/quizzes/[id]/attempts	Submit quiz attempt	A
POST	/v1/simulations/[slug]/attempts	Store lab attempt (computation is client-side)	A
POST	/v1/exercises/[slug]/attempts	Save/evaluate architecture (validation is client-side)	A
POST	/v1/interviews	Create interview session (text or voice mode)	A
POST	/v1/interviews/[id]/responses	Submit interview response	A
GET	/v1/progress	Mastery matrix	A
GET	/v1/quota/today	Today's quota usage for all services	A
GET/POST	/v1/notes	List / create concept-level notes	B
GET/PUT/DELETE	/v1/notes/[id]	CRUD on a single note	B
POST	/v1/notes/[id]/to-flashcard	Convert a note into a review item	B
GET/POST	/v1/concepts/[slug]/annotations	List / create annotations	B
GET/POST	/v1/concepts/[slug]/highlights	List / create highlights	B

Method	Endpoint	Purpose	Tier
GET/POST	/v1/collections	List / create collections	B
GET/PUT/DELETE	/v1/collections/[id]	CRUD on a single collection	B
POST/DELETE	/v1/collections/[id]/items	Add / remove concept from collection	B
POST	/v1/cost/estimate	Calculate cost (uses bundled pricing JSON, no API call)	B
GET	/v1/cost/estimates	List recent cost estimates	B
GET	/v1/cost/pricing/as-of	Get pricing dataset version + as-of date	B
GET/PUT	/v1/career/path	Get / set target role	B
GET	/v1/career/gap-analysis	Current gap analysis vs target role	B
POST	/v1/career/plan	Generate 4-week gap-closing plan	B
POST	/v1/voice/sessions	Start TTS session (metadata only; audio is client-side)	C
POST	/v1/voice/sessions/[id]/position	Update playback position	C
POST	/v1/voice/interviews/[id]/transcript	Save STT transcript (client-side STT)	C
POST	/v1/mentor/sessions	Start AI mentor session (quota-guarded)	C
GET	/v1/mentor/sessions/[id]	Get mentor session result	C
DELETE	/v1/mentor/sessions/[id]	Delete AI session history	C
WS	/v1/collab/[session_id]	WebSocket for CRDT sync + presence (Durable Object)	B
POST	/v1/collab/sessions	Create a collab session	B
GET	/v1/collab/sessions/[id]	Get session metadata + snapshot URL	B
POST	/v1/collab/invite	Generate joinable invite	B
POST	/v1/collab/sessions/[id]/snapshot	Force-save CRDT snapshot to R2	B

7. Example Concept Response (Static JSON)

Served from Cloudflare Pages as a static JSON file. **Zero Worker requests, zero D1 queries.** The user's mastery for this concept is fetched separately from the Worker.

```
{
  "slug": "cache-aside",
  "version": 4,
  "title": "Cache Aside",
  "phase": "Caching",
  "estimated_minutes": 12,
  "difficulty": "core",
  "prerequisites": ["caching-strategies", "latency-vs-throughput"],
  "blocks": [
    {"type": "prose", "id": "why", "payload": {...}},
    {"type": "diagram", "id": "flow-1", "payload": {...}},
    {"type": "simulation", "id": "cache-lab", "payload": {...}},
    {"type": "code_walkthrough", "id": "code-1", "payload": {
        "pattern_slug": "cache-aside-python",
        "language": "python"
    }},
    {"type": "quiz", "id": "q-123", "payload": {...}}
  ],
  "cost_metadata": {
    "reference_architecture_hash": "abc123...",
    "reference_monthly_cost_usd": 340,
    "dominant_cost_driver": "ElastiCache node hours",
    "reference_scale": "10k QPS, 1M users",
    "pricing_dataset_version": "aws-2026-08"
  },
  "voice_alt_text": {
    "flow-1": "The diagram shows a client making a request to the service, which first checks the
cache. On a hit, it returns the cached value. On a miss, it fetches from the database, writes to the
cache with a TTL, and returns."
  },
  "code_pattern_refs": [
    {"slug": "cache-aside-python", "language": "python"},
    {"slug": "cache-aside-go", "language": "go"}
  ],
  "interview_prompts": [...],
  "misconceptions": [...],
  "mastery_rules": {...},
  "version": 4,
  "status": "published"
}
```

Static concept JSON (served from Pages, no Worker).

8. Mastery Algorithm

Tier A MVP (v0.1, v0.5): 5-dimension mastery. **Tier B (v1.0):** adds Cost and Code dimensions for 7 total. Weights rebalance so Cost + Code together account for 20%. A concept is Mastered only when overall score is above threshold AND at least one application task passed AND (for cost-relevant concepts in v1.0+) at least one cost estimate made within 30% of reference.

```
TIER A MVP (v0.1, v0.5) - 5 dimensions:

mastery = 0.30*recall + 0.30*apply + 0.15*explain + 0.15*quiz + 0.10*interview

status:
    mastered         if mastery >= 0.85 and apply >= 0.80
    applied          if apply >= 0.70
    practiced        if quiz >= 0.70 or simulation_pass
    understood       if lesson_complete and quiz >= 0.55
    exposed          if lesson_started
    review_due       if due_at <= now and mastered is false


TIER B (v1.0+) - 7 dimensions:

mastery = 0.25*recall + 0.25*apply + 0.15*explain + 0.15*quiz
        + 0.10*interview + 0.05*cost + 0.05*code

status:
    mastered         if mastery >= 0.85 and apply >= 0.80
                          and (cost not relevant OR cost >= 0.60)
    applied          if apply >= 0.70
    practiced        if quiz >= 0.70 or simulation_pass
    understood       if lesson_complete and quiz >= 0.55
    exposed          if lesson_started
    review_due       if due_at <= now and mastered is false

Notes:
- "cost relevant" is a per-concept flag in cost_metadata (static JSON)
- "code relevant" is a per-concept flag based on code_pattern_refs (non-empty)
- If neither cost nor code is relevant, those weights redistribute to recall+apply
```

Mastery algorithm. Tier A: 5 dims. Tier B: 7 dims.

9. Recommendation Engine

Deterministic and explainable. Tier A uses 5 signals. Tier B (v1.0) adds career_gap and cost_weakness signals. Every recommendation surfaces its top-3 contributing signals so the learner understands WHY this concept, not just WHAT next.

```
TIER A MVP (v0.1, v0.5):

candidate_score =
    prerequisite_ready * 0.30 +
    roadmap_priority   * 0.25 +
    weakness_score     * 0.20 +
    review_due         * 0.15 +
    user_interest      * 0.10
```

```
TIER B (v1.0+):

candidate_score =
    prerequisite_ready * 0.25 +
    roadmap_priority   * 0.20 +
    weakness_score     * 0.18 +
    review_due         * 0.12 +
    user_interest      * 0.08 +
    career_gap         * 0.10 +   # NEW
    cost_weakness      * 0.07       # NEW

pick top candidate with a diversity rule
so the user does not receive five database topics in a row.

Explainability: each recommendation surfaces its top-3 contributing signals
so the learner understands WHY this concept, not just WHAT next.
```

Recommendation scoring. Deterministic + explainable.

10. Spaced Review Scheduling

Simplified SM-2-like state. Track repetitions, interval, ease, and due date. v2.1 adds support for flashcards converted from notes and highlights (Tier B) — they use the same scheduler but carry a different source_type for analytics.

Quality	Interpretation	Action
0-1	Forgot / incorrect	Reset interval
2	Hard recall	Short interval
3	Good recall	Normal interval
4	Easy recall	Longer interval
5	Very strong recall	Long interval + increase ease

11. Quiz Scoring

· Single choice: exact match.

· Multi-select: partial credit only if explicitly configured.

· Ordering: weighted by position distance or exact order depending item.

· Prediction: deterministic expected range or state is preferred; AI explanation is secondary (and quota-guarded).

· Scenario: rubric-based deterministic criteria first.

· **[Tier B, v1.0]** Cost reasoning: estimate within 30% of reference = full credit; within 50% = partial; else no credit. Always show the reference breakdown after answering.

· **[Tier B, v1.0]** Code pattern recognition: correct pattern ID = full credit; identifying the right pattern family but wrong variant = partial credit.

· **[Tier C, v2.0]** Voice explanation: AI-scored on 3 dimensions (coverage, clarity, accuracy). Score is 0-1 per dimension; final = weighted average. Quota-guarded; fallback is text-based self-assessment.

12. Architecture Evaluator Rules

The evaluator runs entirely client-side. Zero Worker requests per validation. AI explanation of findings is Tier C (quota-guarded).

Rule	Example finding	Severity
SPOF	Single database with no failover in a HA exercise	High

Rule	Example finding	Severity
Public exposure	Database connected directly to public client	Critical
Missing async boundary	Heavy video processing directly on request path	Medium/High
Hotspot	Single shard receives 80% of modeled traffic	High
Excessive dependency depth	Request traverses 9 synchronous services	Medium
Cache misuse	Cache sits in front of source without invalidation strategy	Medium
Wrong primitive	Pub/sub used when durable single-consumer work is required	Medium
Missing cost annotation [B]	Architecture has no cost estimate attached at submit time	Low (warning only)
Cost outlier [B]	Architecture cost > 3x reference cost for same exercise	Medium (informational)

13. AI Mentor Backend [Tier C]

AI sessions use Cloudflare Workers AI with the free 10k neurons/day allocation. Every request is wrapped in a quota guard. When quota is exhausted, the mentor UI shows "AI quota reached for today - authored explanation below" and falls back to deterministic content. The learner sees current quota usage in the mentor panel.

· Workers AI binding in wrangler.toml: AI = binding to cloudflare workers ai.

· Models: @cf/meta/llama-3.1-8b-instruct (fast, free-tier-friendly) for explanations; @cf/qwen/qwen1.5-14b-chat-awq for deeper reasoning (optional).

· Quota guard wraps every AI call: check daily usage, fail-closed if over 95% of 10k neurons.

· Timeout: 15s per request. Bounded retry (1 retry on transient error).

· Context size limit: max 4k input tokens. Curated context from content model, not arbitrary DB dumps.

· Cache identical explanatory prompts in KV (24h TTL, prompt-hash key).

· Deterministic fallback: authored explanation from content model, or "AI quota reached - try again after midnight UTC".

· AI never mutates mastery scores. AI can only suggest.

· AI never generates cost estimates. It can only explain the output of the deterministic cost engine (which uses bundled pricing).

· AI cannot read private notes unless the user explicitly shares them in the session.

· All AI calls logged in quota_usage table (service = "ai_neurons", count = neurons used).

· User can delete AI session history at any time. Deletion is irreversible.

14. External Resource Service [Tier C]

```
LinkPreviewAdapter (Tier C)
  fetch(url)          # Worker fetch with 5s timeout
  normalize(content)  # Extract title, description, og:image
  cache(key, ttl=7d)  # KV cache
  fallback(url)       # Return {url, title: url, description: null, image: null}

Rules:
  timeout <= 5s
  cache in KV (7d TTL)
  fail closed -> return URL-only preview
  never block concept content (link previews are optional)
  max 10 previews fetched per day (self-imposed to conserve Worker quota)
```

Link preview adapter. KV cache + fail-closed fallback.

15. Service Adapters (v2.1)

v2.1 has far fewer adapters than v2.0. Most "external" services are now browser-native (TTS, STT) or bundled (cloud pricing) or deferred (code execution, calendar). The remaining adapters are: Workers AI, Link Preview, and the Durable Object client for collab.

15.1 Workers AI Adapter [Tier C]

```
WorkersAIAdapter
  generate(prompt, model, max_tokens) -> str
  count_neurons(prompt, response) -> int
  check_quota() -> {used, limit, remaining}

Implementation:
  - Uses Cloudflare Workers AI binding (AI binding in wrangler.toml)
  - Models: llama-3.1-8b-instruct (default), qwen1.5-14b (optional, deeper)
  - Quota guard wraps every call (see /core/quota_guard.py)
  - KV cache for identical prompts (24h TTL, prompt-hash key)
  - Fallback: deterministic authored explanation

Rules:
  max 4k input tokens
  max 1k output tokens
  15s timeout
  1 retry on transient error
  fail-closed at 95% of 10k neurons/day
```

Workers AI adapter with quota guard and KV cache.

15.2 Voice (Browser-Native, No Server Adapter)

```
Voice is handled entirely client-side via Web Speech API.
No server adapter needed.

TTS (Text-to-Speech):
  const utterance = new SpeechSynthesisUtterance(text);
  utterance.voice = speechSynthesis.getVoices().filter(v => v.lang === 'en-US')[0];
  utterance.rate = 1.0;
  speechSynthesis.speak(utterance);

  Browser support:
    Chrome (macOS/Windows/Android): excellent
    Safari (macOS/iOS): good
    Firefox: limited
    Linux: depends on installed voices (espeak, festival)

  Fallback: show text version alongside audio.

STT (Speech-to-Text):
  const recognition = new SpeechRecognition();
  recognition.lang = 'en-US';
  recognition.continuous = true;
  recognition.interimResults = true;
  recognition.onresult = (event) => { /* transcript */ };
  recognition.start();

  Browser support:
    Chrome: excellent
```

```
    Edge: excellent
    Safari: partial (iOS 14.5+)
    Firefox: NOT supported

  Fallback: text input mode.

Server-side:
  - /v1/voice/sessions stores session metadata (concept, voice choice, playback position)
  - /v1/voice/interviews/[id]/transcript stores transcript text only (no audio)
  - Zero server-side TTS/STT processing
  - Zero cost
```

Voice via Web Speech API. Zero server cost.

15.3 Cost Calculator (Bundled Pricing)

```
CostCalculator (Tier B)
  estimate(architecture_graph, cloud, region) -> CostEstimate
  compare(architecture_graph) -> dict[cloud, CostEstimate]

Pricing data source:
  content/pricing/aws.json    (bundled, versioned in git)
  content/pricing/gcp.json    (later)
  content/pricing/azure.json  (later)

  Schema:
    {
      "version": "aws-2026-08",
      "as_of": "2026-08-01",
      "services": [
        {
          "service_code": "ec2.m6i.xlarge",
          "name": "EC2 m6i.xlarge",
          "region": "us-east-1",
          "price_unit": "hour",
          "price_usd": 0.192,
          "pricing_model": "on_demand"
        },
        ...
      ]
    }

  Refresh: quarterly via PR (manual; author checks AWS pricing page)

Rules:
  pricing data is static JSON, served from Pages (zero Worker cost for reads)
  calculation runs client-side (zero Worker cost for calculation)
  estimate saved to D1 only on explicit "Save"
  all estimates include pricing_dataset_version + as_of date
  no live API calls, no surprise bills, works offline

Comparison:
  /v1/cost/compare runs 3 client-side calculations (AWS, GCP, Azure)
  Returns side-by-side dict in < 1s
```

Cost calculator with bundled pricing dataset. Zero API calls.

15.4 Collaboration (Durable Object + Yjs)

```
CollabSessionDurableObject (Tier B, v2.0)
  - One Durable Object per collab session
  - Holds the Yjs document in memory (SQLite-backed by Durable Object storage)
  - WebSocket connections for each participant
  - Broadcasts CRDT updates to all participants
  - Presence (cursor + name) broadcast every 100ms (throttled)

Implementation:
  - Yjs library loaded in Durable Object (pure JS, works in Workers runtime)
  - y-websocket-style server adapter (simplified)
  - Snapshots saved to R2 on:
    - every 60s during active session (via alarm)
    - on session end
    - on owner-initiated "save"

  wrangler.toml binding:
```

```
    [[durable_objects.bindings]]
    name = "COLLAB_SESSION"
    class_name = "CollabSessionDurableObject"

    [[migrations]]
    tag = "v1"
    new_sqlite_classes = ["CollabSessionDurableObject"]

Rules:
  max 4 concurrent participants (personal use, not Miro)
  invite URLs use 32-byte random slugs (unguessable)
  sessions auto-end after 2h of inactivity (alarm)
  all participants get a copy of the final CRDT doc on session end
  no Redis, no Yjs server, no S3 - all Cloudflare free tier
```

Collab via Durable Objects. Yjs CRDT. R2 snapshots.

16. Quota Guard Implementation

The quota guard is the most important piece of v2.1 backend code. It wraps every optional service call (Workers AI, link preview) and every significant Worker/D1/R2/KV operation. When a quota is near limit, it warns. When exhausted, it fails-closed with a deterministic fallback. **Never bills.**

```
# core/quota_guard.py

FREE_TIER_LIMITS = {
    'worker':       100_000,  # requests/day
    'd1_read':      5_000_000,  # rows read/day
    'd1_write':     100_000,  # rows written/day
    'r2_class_a':   1_000_000 / 30,  # per day approximation
    'r2_class_b':   10_000_000 / 30,
    'kv_read':      100_000,
    'kv_write':     1_000,
    'do_request':   100_000,
    'ai_neurons':   10_000,
}

async def track_usage(service: str, count: int = 1):
    """Increment daily usage counter in D1."""
    today = datetime.utcnow().strftime('%Y-%m-%d')
    await d1.execute(
        'INSERT INTO quota_usage (date, service, count, last_updated) '
        'VALUES (?, ?, ?, CURRENT_TIMESTAMP) '
        'ON CONFLICT(date, service) DO UPDATE SET '
        'count = count + ?, last_updated = CURRENT_TIMESTAMP',
        [today, service, count, count]
    )

async def get_usage(service: str) -> dict:
    """Get today's usage for a service."""
    today = datetime.utcnow().strftime('%Y-%m-%d')
    row = await d1.first(
        'SELECT count FROM quota_usage WHERE date = ? AND service = ?',
        [today, service]
    )
    used = row['count'] if row else 0
    limit = FREE_TIER_LIMITS[service]
    return {
        'used': used,
        'limit': limit,
        'remaining': max(0, limit - used),
        'percent': round((used / limit) * 100, 1),
    }

async def with_quota_guard(
    service: str,
    request_fn,
    fallback_fn,
    warn_threshold: float = 0.80,
    block_threshold: float = 0.95
):
    """Wrap an optional service call with quota protection."""
    usage = await get_usage(service)
```

```
    if usage['percent'] >= block_threshold * 100:
        return fallback_fn('quota_exhausted')

    try:
        result = await request_fn()
        await track_usage(service, 1)
        return result
    except QuotaExceededError:
        return fallback_fn('quota_exhausted')
    except Exception as e:
        log_error(e)
        return fallback_fn('error')

# Usage:
response = await with_quota_guard(
    'ai_neurons',
    lambda: workers_ai.generate(prompt, model='llama-3.1-8b-instruct'),
    lambda reason: authored_fallback.explain(concept_slug) if reason == 'quota_exhausted'
                   else generic_error_response(reason)
)
```

Quota guard implementation. The heart of zero-cost guarantee.

17. Error Model

Code	Meaning	Client behavior
400	Invalid request	Show field-level guidance
401	Unauthenticated	Refresh session / sign in via GitHub OAuth
403	Unauthorized (not the allowed GitHub user)	Show access message
404	Unknown resource	Not found state
409	Version/conflict	Refresh and retry
422	Semantic validation failure	Explain invalid action
429	Rate limited	Retry with backoff
500	Unexpected server error	Friendly fallback + request id
503	Optional dependency unavailable	Use degraded mode (e.g., text mode if TTS unavailable)
504 [NEW]	AI quota exhausted / timeout	Show "AI quota reached for today" + authored fallback
510 [NEW]	Quota guard triggered	Show "Daily quota for [service] reached - try again after midnight UTC"

18. Background Worker Contract

Cloudflare Queues + Cron Triggers replace Celery/RQ. Jobs are idempotent and bounded-retry. Free tier: 10k operations/day, 24h retention.

```
Task(payload, idempotency_key)
  validate(payload)
  execute()
  record_result()
  retry on transient errors only (max 3)
  dead-letter after bounded retries

New v2.1 tasks:
  QuotaResetTask          (Cron: midnight UTC)
  QuotaWarningCheckTask   (Cron: every hour)
  ReviewSchedulingTask    (Cron: nightly reconciliation)
  AnalyticsAggregationTask (Cron: hourly)
  WeeklyDigestTask        (Cron: Sunday 9pm user-local)
  PricingRefreshCheckTask (Cron: monthly - reminds author to PR update)
  CostEstimateRevalidationTask (on pricing dataset version change)
  ExportGenerationTask    (user request via Queue)
  AIMentorTask            (user request via Queue, quota-guarded)
  VoiceTranscriptSaveTask (on voice interview session end)
```

```
  CollabSnapshotTask      (every 60s via Durable Object alarm)
  StreakRecoveryTokenTask (Cron: daily check)
```

Worker contract. Cloudflare Queues + Cron Triggers.

19. Admin and Content Pipeline

There is no admin panel. Content edits are PRs to the content/ directory in git. CI runs validation on every PR. This eliminates an entire class of admin auth, roles, and runtime mutation concerns.

1. Draft concept (edit JSON in content/concepts/[area]/[slug].json).

2. CI validates schema (Zod schemas in packages/content-schema/).

3. CI runs content checks: broken links, missing prerequisite refs, duplicate IDs, missing quiz rationale, missing voice alt-text for diagrams, missing cost_metadata for cost-relevant concepts.

4. CI previews rendered lesson via Pages preview deployment.

5. Reviewer (you) approves PR.

6. Merge to main triggers deploy.

7. Pages rebuilds static bundles.

8. Worker cache invalidation via Cache API (if needed).

9. Search index (Tier B FTS5) rebuilt via Cron on next run.

10. If cost_metadata changed, CostEstimateRevalidationTask runs on next Cron.

20. Backend Testing Matrix

Test type	Examples
Unit	Mastery (5-dim and 7-dim), scheduler, scoring (incl. cost + code + voice), recommendations, validation rules, cost calculator, quota guard logic. Run in Node (not Pyodide) for speed.
Integration	Content retrieval (static), progress write, quiz submission, notes CRUD, collections CRUD, cost estimate, collab session lifecycle. Run against D1 local emulator (wrangler d1 local).
E2E	Onboarding -> daily dose -> lesson -> quiz -> review (text mode), architecture playground -> cost estimate -> case study submit, collab session create -> invite -> join -> concurrent edit -> snapshot. Run in Cloudflare Workers local emulator (wrangler dev).
Quota guard	Simulate quota exhaustion for each service. Verify fail-closed behavior. Verify deterministic fallbacks work. Verify no bill-generating path exists.
Adapter fallback	Simulate Workers AI quota exhaustion, link preview timeout, browser TTS unavailability. Verify graceful degradation. Verify UI shows correct fallback message.
Security	Auth boundaries (single-user allow-list), rate limits, injection, OAuth scope verification, collab invite URL unguessability.
Performance	Home endpoint (< 100ms), concept reads (static, < 50ms edge), event batch ingestion (< 200ms for 20 events), cost estimate calculation (< 500ms client-side), quota dashboard query (< 50ms).
Migration	Forward migration + representative seed restore. Verify new tables are nullable + have defaults.

21. Seed Data Strategy

Ship a curated seed dataset with the first 25-40 concepts, several question types, one simulation per major family, 5-7 complete case studies, 3 code patterns per major family (Python + Go), 1 reference cost estimate per case study, 4 system starter collections, and the bundled AWS pricing dataset (~20 services). All static JSON in git. D1 seeded with: 1 user row, empty mastery, empty review_items, empty attempts, quota_usage with today's date and zero counts.

22. Backend Build Order

#	Step	Tier	Version
1	Cloudflare Worker setup (wrangler.toml, D1 binding, GitHub OAuth)	A	0.1
2	D1 schema v0.1 (users, mastery, review_items, learning_events, attempts, interview_sessions, achievements, user_achievements, quota_usage)	A	0.1
3	Quota guard module + /quota/today endpoint	A	0.1
4	Auth (GitHub OAuth, single-user allow-list, JWT in httpOnly cookie)	A	0.1
5	Home endpoint (daily dose, review queue, recommendations - 5 signals)	A	0.1
6	Quiz + attempts + scoring (5-dim mastery)	A	0.1
7	Review scheduler (SM-2-like, Cron nightly)	A	0.1
8	Progress dashboard endpoint (mastery matrix)	A	0.1
9	Static content pipeline (JSON in git -> Next.js static props -> Pages)	A	0.1
10	Simulation attempts (computation client-side; Worker stores result)	A	0.5
11	Architecture attempts (validation client-side; Worker stores result + cost_estimate_id later)	A	0.5
12	Case studies (static; attempt tracking in D1)	A	0.5
13	Interview sessions (text mode; Cron for timeout)	A	0.5
14	D1 schema v1.0 (notes, annotations, highlights, collections, collection_items, cost_estimates, career_paths)	B	1.0
15	Notes + annotations + highlights CRUD + flashcard conversion	B	1.0
16	Collections CRUD + system starter packs (static)	B	1.0
17	Bundled pricing dataset (AWS, ~20 services) in git	B	1.0
18	Cost calculator (client-side calc; Worker stores estimate)	B	1.0

#	Step	Tier	Version
19	Career path (role profiles static; gap analysis + plan generation on Worker)	B	1.0
20	Architecture playground extensions (cost annotation + version history + export)	B	1.0
21	Case study extensions (Cost step + Interview Summary step)	B	1.0
22	Mastery expansion to 7 dimensions (cost_score, code_score)	B	1.0
23	D1 schema v2.0 (voice_sessions, voice_interview_attempts, collab_sessions, collab_participants)	C/B	2.0
24	Voice sessions (metadata only; audio is client-side Web Speech API)	C	2.0
25	Voice interview transcripts (STT client-side; Worker stores transcript)	C	2.0
26	Workers AI adapter + quota guard + KV cache	C	2.0
27	AI mentor endpoints (quota-guarded, deterministic fallback)	C	2.0
28	CollabSession Durable Object + Yjs CRDT	B	2.0
29	Collab endpoints (WebSocket + REST + invite)	B	2.0
30	Link preview adapter (Worker fetch + KV cache, Tier C)	C	2.0

23. wrangler.toml Configuration

The wrangler.toml is the single source of truth for all Cloudflare bindings. It declares every free-tier resource the app uses. This is the deployment contract.

```
name = "nocap-worker"
main = "src/main.py"
compatibility_date = "2026-08-16"
compatibility_flags = ["python_workers"]

# Account bound via CLOUDFLARE_ACCOUNT_ID env var (GitHub Actions secret).

# D1 (SQLite) - 5GB free, 5M reads/day, 100k writes/day
[[d1_databases]]
binding = "DB"
database_name = "nocap"
database_id = "<your-d1-id>"

# R2 (Object storage) - 10GB free, free egress
[[r2_buckets]]
binding = "R2"
bucket_name = "nocap-assets"

# KV (Cache) - 1GB free, 100k reads/day, 1k writes/day
[[kv_namespaces]]
binding = "KV"
id = "<your-kv-id>"

# Durable Objects (Tier B, v2.0 - collab) - 100k req/day, 13k GB-s/day
[[durable_objects.bindings]]
name = "COLLAB_SESSION"
class_name = "CollabSessionDurableObject"

[[migrations]]
tag = "v1"
new_sqlite_classes = ["CollabSessionDurableObject"]

# Workers AI (Tier C) - 10k neurons/day free
[ai]
binding = "AI"

# Queues (Tier B/C) - 10k ops/day free
[[queues.producers]]
queue = "nocap-tasks"
binding = "TASKS"

[[queues.consumers]]
queue = "nocap-tasks"
max_batch_size = 10
max_batch_timeout = 5

# Cron Triggers - 5 free per Worker
[triggers]
crons = [
  "0 0 * * *",      # midnight UTC: quota reset
  "0 * * * *",      # hourly: quota warning check
  "0 1 * * *",      # 1am UTC: review scheduling reconciliation
  "0 * * * *",      # hourly: analytics aggregation (same cron, different handler)
  "0 21 * * 0",     # Sunday 9pm UTC: weekly digest
```

```
]

# Observability
[observability]
enabled = true

# Secrets (set via `wrangler secret put`):
# - GITHUB_CLIENT_ID
# - GITHUB_CLIENT_SECRET
# - JWT_SECRET
```

wrangler.toml. All Cloudflare free-tier resources declared.

24. Deployment

v2.1 deployment topology. All on Cloudflare free tier.

Deployment is fully automated via GitHub Actions on push to main. No manual steps. No staging environment (single user). Preview deployments on PRs (Cloudflare Pages preview). Rollback via git revert + redeploy. D1 migrations run via `wrangler d1 migrations apply` in the deploy script.

25. Monthly Cost Summary

Service	Free Tier	Expected Usage (single user)	Monthly Cost
Cloudflare Workers	100k req/day	~5-15k/day	Rs 0
Cloudflare D1	5GB, 5M reads/day, 100k writes/day	< 100MB, < 100k reads/day, < 5k writes/day	Rs 0
Cloudflare R2	10GB, 1M+10M ops/month, free egress	< 1GB storage, minimal ops	Rs 0

Service	Free Tier	Expected Usage (single user)	Monthly Cost
Cloudflare KV	1GB, 100k reads/day, 1k writes/day	< 1k reads/day (link previews only)	Rs 0
Cloudflare Durable Objects	100k req/day, 13k GB-s/day	Only during collab sessions (Tier B v2.0)	Rs 0
Cloudflare Queues	10k ops/day, 24h retention	< 100/day	Rs 0
Cloudflare Workers AI	10k neurons/day	< 2k/day (AI mentor is opt-in)	Rs 0
Cloudflare Pages	Unlimited static, 500 builds/month	1-5 builds/day	Rs 0
Cloudflare Cron Triggers	5 per Worker (free)	5 in use	Rs 0
GitHub (source + CI)	Free for personal repos	Standard usage	Rs 0
Domain (optional)	~Rs 1,000/year (.com)	Optional - can use .pages.dev subdomain	Rs 0-83/mo
		TOTAL	**Rs 0/month**

ZERO-COST GUARANTEE

No paid dependency and no accidental billing. Core functionality remains usable at free-tier exhaustion. Cloudflare's Free plan fails-closed (queries fail rather than silently switching to paid usage), which is the architectural property that makes this guarantee possible. Quota guards enforce graceful degradation. Raw quota data lives under Settings → System Health (not prominent in the main UI).